FinAgent-Bench: A Benchmark for Evaluating Agentic LLMs on Financial Analysis Tasks with Tool Use

Anonymous ACL submission

Abstract

We introduce FinAgent-Bench, a comprehensive benchmark for evaluating the capabilities of large language models (LLMs) in agentic financial analysis tasks requiring tool use. The benchmark consists of two complementary datasets: (1) SynthSQL, a synthetic dataset of 5,803 SQL-based financial queries with committee-reviewed quality control, and (2) GroundedFinQA, a grounded dataset derived from FinQA containing 1,108 multi-hop reasoning tasks with synthetic databases and tool abstractions. Both datasets require models to interact with databases through tool calls rather than relying on in-context information, testing genuine agentic reasoning capabilities. We evaluate a range of frontier and open-source models, finding that state-of-the-art models achieve only 50–68% accuracy on SynthSQL and 53–70% on GroundedFinQA, demonstrating significant room for improvement in agentic financial reasoning. We release the full dataset, evaluation code, and model traces to facilitate future research.

1 Introduction

Large language models (LLMs) are increasingly deployed as autonomous agents capable of using tools to accomplish complex tasks (Schick et al., 2023; Qin et al., 2023b). In the financial domain, such agents could assist with financial analysis, compliance checking, and decision support by querying databases and synthesizing information from multiple sources.

However, evaluating the agentic capabilities of LLMs presents significant challenges. Existing financial QA benchmarks like FinQA (Chen et al., 2021) and TAT-QA (Zhu et al., 2021) primarily test reading comprehension and numerical reasoning over provided contexts, rather than genuine tool-use and multi-step reasoning. Moreover, many benchmarks suffer from data contamination, where test examples may have been seen during model training.

We introduce FinAgent-Bench, a benchmark specifically designed to evaluate agentic LLMs on financial analysis tasks requiring tool use. Our benchmark consists of two complementary datasets:

SynthSQL A synthetic dataset of SQL-based financial queries where agents must interact with dynamically generated databases through an `execute_sql` tool. Each example undergoes rigorous quality control through a committee of three LLM judges.

GroundedFinQA A grounded dataset derived from FinQA where each sample includes a complete agentic environment with diverse custom tools, synthetic databases, and distractor tools and data designed to test multi-hop reasoning.

Both datasets share key design principles: (1) answers cannot be extracted from the prompt alone—agents must use tools to retrieve information, (2) examples require multi-step reasoning with intermediate tool calls, and (3) distractor information is present to test agents' ability to filter relevant from irrelevant data.

Our main contributions are:

1. A novel benchmark for evaluating agentic LLMs on financial tasks requiring genuine tool use.
2. A rigorous dataset creation pipeline with LLM committee-based quality control.
3. Comprehensive evaluation of frontier and open-source models revealing significant gaps in agentic reasoning capabilities.
4. Public release of datasets, evaluation code, and model traces to support reproducible research.

2 Related Work

FinAgent-Bench is situated at the intersection of three research areas: context-bound financial Question Answering (QA), general agentic tool-use evaluation, and domain-specific semantic parsing.

Context-Bound Financial QA. Foundational financial benchmarks like FinQA (Chen et al., 2021) and TAT-QA (Zhu et al., 2021) measure reading comprehension and numerical reasoning over static, explicitly provided contexts. These established datasets are inherently non-agentic as they do not require dynamic interaction or external system calls. Our GroundedFinQA component transitions this evaluation to a dynamic environment where information must be actively retrieved through multi-hop tool calls, isolating and testing the core planning and execution logic.

Agentic Reasoning and Tool Use. General agent benchmarks, such as ToolBench (Qin et al., 2023a) and MCP-Verse (Lei et al., 2025), establish the methodological standard for evaluating LLMs' planning abilities against a large breadth of tools and domains. While essential for assessing general tool augmentation, these benchmarks prioritize scale over domain depth. Financial analysis critically demands precision and specialized interpretation. By specializing the tool environment for finance, FinAgent-Bench tests the *depth* of financial logic, revealing significant gaps where general planning skills fail to confer the necessary domain competence (e.g., 13–59% accuracy on grounded financial reasoning tasks). Recent real-world financial benchmarks like FinGAIA (Zeng et al., 2025) and the Finance Agent Benchmark (Bigeard et al., 2025) prioritize realism by using live web data (SEC EDGAR), while FinAgent-Bench is engineered for **diagnostic fidelity** using controlled, synthetic environments to attribute failures unequivocally to flawed planning logic, rather than external noise or API variability.

Text-to-SQL Semantic Parsing. The Text-to-SQL task, which involves translating natural language into executable SQL, is central to structured data access. Cross-domain datasets like Spider (Yu et al., 2018) and BIRD (Li et al., 2024) primarily evaluate the fidelity of the generated SQL string (semantic parsing accuracy). In contrast, FinAgent-Bench's SynthSQL component transforms this into an *end-to-end agentic execution challenge*. The agent must not only generate the correct query but also successfully call the `execute_sql` tool and synthesize the retrieved result into a final, natural language answer. This emphasis on the full execution sequence moves the task beyond purely linguistic parsing and tests the agent's ability to integrate data retrieval into multi-step reasoning.

Our approach leverages synthetic data generation and a rigorous quality control process, utilizing a committee of three LLM judges, to mitigate data contamination risks and ensure high integrity at scale, validating that observed low accuracy reflects genuine performance limitations.

Financial QA Benchmarks. Several benchmarks exist for evaluating financial reasoning in LLMs. FinQA (Chen et al., 2021) and ConvFinQA (Chen et al., 2022) test numerical reasoning over financial reports. TAT-QA (Zhu et al., 2021) combines tabular and textual reasoning. FinanceBench (Islam et al., 2023) provides questions requiring reasoning over SEC filings. However, these benchmarks provide all necessary information in context, testing reading comprehension rather than tool use.

Agentic Benchmarks. Recent work has introduced benchmarks for evaluating LLM agents. AgentBench (Liu et al., 2023) tests agents across diverse environments including web browsing and code execution. ToolBench (Qin et al., 2023b) evaluates tool use across thousands of APIs. SWE-bench (Jimenez et al., 2024) tests agents on real-world software engineering tasks. However, none of these focus specifically on financial analysis with database tools.

Synthetic Data Generation. Synthetic data generation has proven valuable for creating evaluation benchmarks at scale (Wang et al., 2022). Recent work has explored using LLMs to generate and verify synthetic datasets with quality control mechanisms (Prabhakar et al., 2025). Our approach utilizes these methods and adapts them to creating financial agentic datasets.

3 Dataset Description

3.1 Overview

FinAgent-Bench comprises two complementary datasets targeting different aspects of agentic financial reasoning. Table 2 summarizes their key characteristics.

Benchmark	Financial Focus	Evaluation Focus	Agentic Requirement
FinQA / TAT-QA	High	Static Reading Comprehension	Low
Spider / BIRD	Low/Medium	Semantic Parsing (SQL Fidelity)	Low
FinGAIA / Finance Agent	High	Real-time Retrieval / Realism	High
FinAgent-Bench	**High**	**Diagnostic Execution / Tool Sequence**	**High**

Table 1: Comparison of key benchmarks.

Property	SynthSQL	GroundedFinQA
# Examples	5,979	1,108
Tool Type	SQL queries	Custom functions
Multi-hop	$\geq$ 1 hop	$\geq$ 2 hops
Distractor data	Yes	Yes
Quality control	Committee	Execution-based

Table 2: Overview of the two datasets in FinAgent-Bench.

3.2 SynthSQL: Synthetic SQL Dataset

SynthSQL consists of financial analysis queries that require SQL database interaction. Each example contains:

User role The persona making the query (e.g., CFO, auditor, financial analyst).

Query A natural language question about financial data.

Database schema A plausible company database schema.

Database contents Realistic financial data including distractor rows.

Expected answer The ground-truth answer.

Agent trace A reference solution demonstrating correct tool use.

The dataset spans diverse financial analysis categories including:

- Accounts Payable (AP) analysis
- Revenue recognition
- Variance analysis
- Data integrity and reconciliation
- Financial reporting

Example Query.

> "Which invoices from our most important suppliers are more than 90 days overdue?"

To answer this query, an agent must: (1) identify which suppliers are "most important" (e.g., by spend volume), (2) find invoices from those suppliers, and (3) filter by payment status and age.

3.3 GroundedFinQA: Grounded FinQA Dataset

GroundedFinQA transforms FinQA samples into complete agentic environments. Each example includes:

System prompt Scenario description and available tools (without table data).

Synthetic database SQLite (Hipp, 2024) database modeling the scenario.

Core tools Functions required to derive the answer.

Distractor tools Irrelevant tools to test filtering ability.

Correct plan Reference multi-hop solution.

Critically, the agent receives only the scenario description and tool definitions—not the underlying data table. This ensures agents must use tools to retrieve information rather than extracting answers from context.

Example Item. An example from the GroundedFinQA dataset is shown in Table 3, truncated for brevity. The full example contains a longer system prompt, database contents, and fully runnable Python code for the correct solution.

4 Dataset Creation

4.1 SynthSQL Creation Pipeline

The SynthSQL dataset is created through a 9-stage pipeline. See Figure 1 for the general composition of the pipeline.

Stage 1: Query Generation. We start with a few seed queries spanning financial analysis categories (in our experiment, we took 12 seed queries). These seeds are then expanded using an LLM, varying user roles and specific details. We utilize a prompt that takes one seed and generates 20 queries to increase diversity and run it multiple times to generate a large number of queries. We also use embedding-based similarity filtering (cosine threshold 0.9) to remove near-duplicates.

Query (part of system prompt)

What is the percent change in receivables from or (payables to) the Money Pool from 2001 to 2002?

Available tools

```
list_available_years()
get_quarterly_money_pool_amount(year, quarter)
get_total_money_pool_amount_for_year(year)
compute_percent_change(old_value, new_value)
get_cash_flow_transactions(start_date, end_date, limit)
sum_cash_flow_for_year(year)
```

Correct tool call sequence

```
def annual_from_quarters(year:  int) -> float:
    return sum(get_quarterly_money_pool_amount(year, q) for q in range(1,
5))
amt_2001 = annual_from_quarters(2001)
amt_2002 = annual_from_quarters(2002)
pct_change = compute_percent_change(amt_2001, amt_2002)
answer = f"round(pct_change)%"
```

Table 3: An example from the GroundedFinQA dataset (excerpt).

Stage 2: Schema Generation. For each query, an LLM generates a plausible database schema that is: (1) sufficient to answer the query, (2) realistic for a company database, and (3) appropriately complex.

Stage 3: Data Generation. The LLM is then tasked with generating realistic data for each query and schema, including: (1) data necessary to answer the query, and (2) distractor rows to increase difficulty. The LLM is also required to provide the correct answer to the query consistent with the generated data.

Stage 4: SQL Validation. All generated SQL (schema and data) is validated by execution. Errors are iteratively fixed using LLM assistance (up to 5 attempts). We found that LLMs struggle with fixing small errors in (relatively) long SQL programs, so we split the program into individual statements and fix them one by one.

Stage 5: Agent Trace Generation. An agent runs on each query. It is supplied with access to the data using the `execute_sql` tool, with up to 6 rounds of interaction allowed. The full conversation trace is recorded.

At this point, we have a dataset that contains:

- A query
- A database schema
- Database contents (data necessary to answer the query and distractor rows)
- Intended answer to the query
- An agent trace demonstrating the solution to the query

However, there may be hallucinations, unnatural data, wrong tool calls, or wrong answers in the agent trace that need to be identified and fixed.

Stage 6: Committee Judgement. A committee of three LLM judges evaluates each example's data and agent trace on five criteria:

Data naturalness Is the generated data realistic?

Trace reasonableness Does the agent examine appropriate data?

Trace soundness Is the reasoning logical?

Grounding Is reasoning based only on tool outputs?

Answer correctness Does the answer address the query?

An example passes only if it receives majority approval (2/3 judges) on *all* criteria. In that case, it proceeds to the next stage. Otherwise, it goes to the improvement loop (Stages 7 and 8). Note that the loop only runs once for each example; if the improved example fails the judgement again, it is rejected completely.

Stage 7: Feedback Reconciliation. The judges' criticisms are given to an LLM which is asked to aggregate them into actionable feedback. The LLM at this stage has access to most of the example's data, including the query, the database schema, the database contents, and the agent trace.

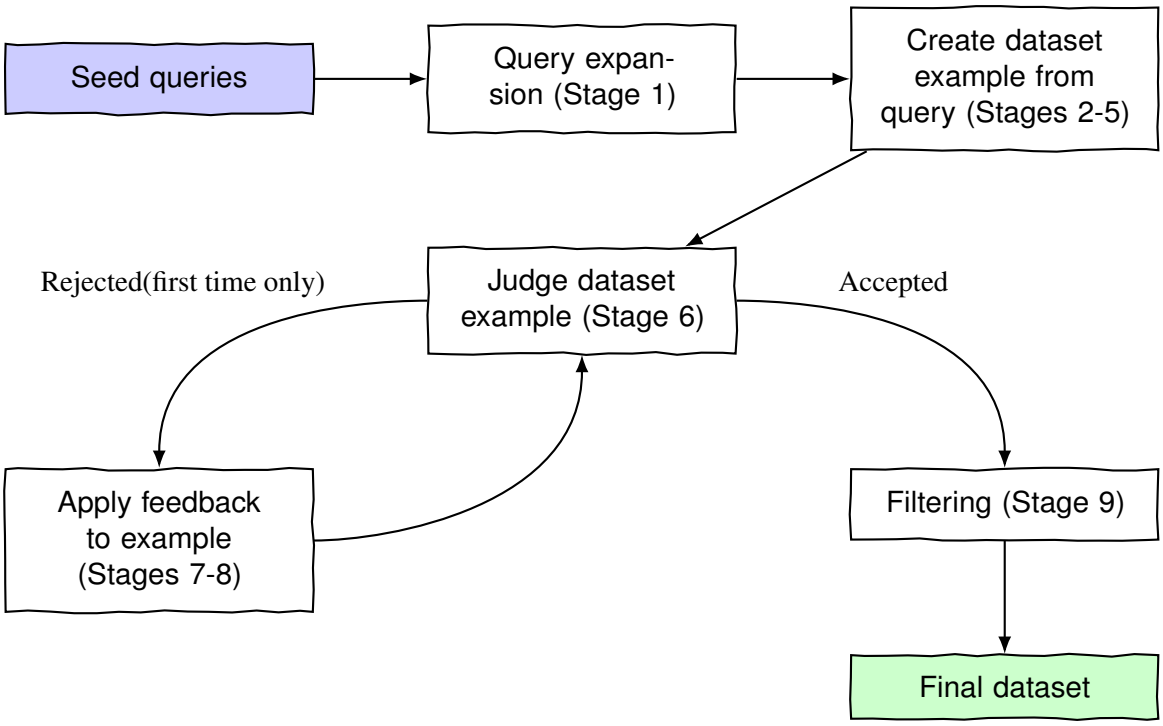

Figure 1: The SynthSQL creation pipeline.

Stage 8: Feedback Application. The agent runs again with a prompt containing:

- The query
- The database schema
- The previous agent trace
- The feedback produced in Stage 7

It still has access to the `execute_sql` tool, and the conversation is limited to 6 rounds. The full conversation trace is recorded.

Stage 9: Second Judgement. The improved examples are then re-judged by the committee. If an example passes, it proceeds to the next stage. Otherwise, it is dropped from the dataset.

Final Filtering. The final dataset includes only examples that passed committee judgement. We run two filtering passes to ensure quality and complexity of the dataset. The first pass uses an LLM to compare the final answer produced in the agent trace with the intended answer created in Stage 3. Additionally, we filter out examples where the agent did not actually access the SQL tool (removing examples answerable from context alone).

4.2 GroundedFinQA Creation Pipeline

The GroundedFinQA pipeline transforms FinQA samples through 9 steps:

Step 1: Initial Solution. An LLM creates a direct Python solution using the original table data, establishing the computational logic.

Step 2: Database Generation. A SQLite database is generated to model the scenario, containing tables needed to answer the question.

Step 3: Basic Tools. Tool functions are created as abstractions over the database, designed to enable multi-hop reasoning.

Step 4: Correct Plan (Basic). A reference solution is written using the basic tools.

Step 5: Augmented Database. The database is expanded with additional tables, split data, and distractor rows to increase difficulty.

Step 6: Augmented Tools. Extended tools are created including:

- Core tools for the correct solution
- Partial information tools requiring chaining
- Distractor tools providing irrelevant data

Step 7: Correct Plan (Augmented). A reference solution using the augmented tools demonstrates the multi-hop solution path.

Step 8: System Prompt. A system prompt is generated containing the scenario description and tool definitions, explicitly excluding table data and answer hints.

Step 9: Agent Code. A runnable agent using the `smolagents` (Roucher et al., 2025) library is generated for evaluation.

Each step is required to output a file (Python code in all steps except Step 8 which outputs a system prompt with the task formulation). For each step, the LLM receives in the context:

- A long description of the whole process with all the steps described,
- An indication of what step is currently running,
- All previous files' content from the previous steps, and
- In Step 9, an example agent code showing coding practices for achieving more stable results.

After completing, each step undergoes an execution-based validation. If the execution fails, the step is rerun with a prompt that includes the error message and asks to fix the errors (up to 10 attempts).

5 Experiments and Evaluation

5.1 SynthSQL Experiments

We created a synthetic dataset of 5,979 examples by starting with 12 seed queries and generating 10,000 diverse queries. We then proceeded with all the remaining stages of the pipeline. The default LLM used in all stages is GPT-4.1-mini with the following exceptions: Stages 4 (SQL validation), 7 (feedback reconciliation), and 8 (feedback application) are run with o4-mini; Stage 6 (committee judgement) is run with three different models. See Table 4 for the number of examples in the dataset after each stage.

Stage	Number of Examples
Seed queries	12
1. Query expansion	10,000
2. Schema generation	10,000
3. Data generation	10,000
4. SQL validation	10,000
5. Agent trace generation	9,560
6. Committee judgement	
– passed	5,401
– to be improved	4,156
7. Feedback reconciliation	4,156
8. Feedback application	3,967
9. Committee judgement	2,832
Total passed so far	8,233
10. Filtering	**5,803**

Table 4: Number of examples in the SynthSQL dataset after each stage.

5.2 GroundedFinQA Experiments

We ran the data example creation routine using the protocol described above for 1,247 FinQA examples, resulting in 1,108 examples (the rest failed in at least one of the stages). Each final example contains the following files:

`initial_solution.txt` The correct answer to the question.

`synthetic_db_generator_augmented.py` Python code that creates the SQLite database.

`tools_augmented.py` The tools required for the multi-step processing, plus distractor tools.

`tool_proxy.py` Same tools adapted for use with `smolagents`.

`correct_plan_augmented.py` The intended sequence of tool calls that solves the task.

`agent_system_prompt.txt` LLM prompt with context, tool descriptions, and the question.

`agent.py` The agent code that runs a specified LLM with the system prompt and tools.

Intermediate files Files created during the example building process.

5.3 Native Tool Calling vs. ReAct

While analyzing the traces of smaller models we found it hard to understand what the models are struggling with when calling tools because there was no trace of the reasoning behind the tool and argument choices. We constructed an additional experiment using the ReAct tool calling protocol (Yao et al., 2022). ReAct specifically requires the model to output its reasoning before calling a tool. This experiment was performed on the SynthSQL part of the dataset.

5.4 Evaluated Models

We evaluate a diverse set of models:

Frontier Models.

- GPT-4.1, GPT-4.1-mini (OpenAI)

Model	Accuracy (%)
Frontier Models	
GPT-5	**68.9**
GPT-5-mini	65.8
o4-mini	67.1
GPT-4.1	62.4
GPT-4.1-mini	61.5
Open-Source Models	
Qwen3-30B-A3B	**50.5**
Qwen3-8B	47.6
Llama-3.1-8B-Instruct	21.9

Table 5: Evaluation results on SynthSQL.

Model	Accuracy (%)
Frontier Models	
GPT-5	**69.6**
GPT-5-mini	67.5
o4-mini	67.3
GPT-4.1	60.6
GPT-4.1-mini	56.9
Open-Source Models	
Qwen3-30B-A3B	53.0
Qwen3-8B	44.1
Llama-3.1-8B-Instruct	16.3

Table 6: Evaluation results on GroundedFinQA.

- GPT-5, GPT-5-mini, o4-mini (OpenAI, reasoning models)

Open-Source Models.

- Qwen3-8B, Qwen3-30B-A3B (Alibaba)
- Llama-3.1-8B-Instruct (Meta)

5.5 Metrics

We use **accuracy** as the primary metric. For SynthSQL, an LLM judge compares the agent's final answer against the expected output and outputs a yes/no verdict. For GroundedFinQA, the numerical answers are compared against the correct plan output with a tolerance of one least significant digit. To save time and reduce costs, for SynthSQL we only evaluate on a randomly selected test set of 729 examples.

6 Results

6.1 SynthSQL Results

Table 5 presents evaluation results on the SynthSQL dataset.

Key Findings.

1. **Significant gap from ceiling**: Even the best model (GPT-5) achieves only 68.9% accuracy, indicating substantial room for improvement.
2. **Model capacity impact**: More modern and capable models, like GPT-5, produce much better results than smaller and older models, like Llama-3.1-8B.
3. **Large open-source gap**: Open-source models significantly underperform frontier models, with even Qwen3-30B-A3B scoring 11 percentage points lower than the worst frontier model (GPT-4.1-mini).

Model	Accuracy (%)	
	Native	**ReAct**
Non-thinking Models		
GPT-4.1	62.4	<u>63.8</u>
GPT-4.1-mini	61.5	<u>63.5</u>
Llama-3.1-8B-Instruct	21.9	<u>28.3</u>
Thinking Models		
GPT-5	<u>68.9</u>	64.6
o4-mini	<u>67.1</u>	64.3
Qwen3-30B-A3B	<u>50.5</u>	46.2
Qwen3-8B	<u>47.6</u>	44.3

Table 7: Evaluation results on tool calling protocol. Underlined values are the best result for each model.

6.2 GroundedFinQA Results

Table 6 presents results on GroundedFinQA.

Key Findings.

1. **Similar to SynthSQL** in difficulty: Best performance is 69.6% (vs. 68.9%), with comparable scores for almost all models.
2. **Consistent model ranking**: GPT-5 and o4-mini lead, followed by GPT-4.1 variants, then open-source models.
3. **Modern models use tools better**: Llama-3.1-8B severely underperforms the newer Qwen3-8B despite having the same parameter count.

6.3 Native Tool Calling vs. ReAct Results

Table 7 presents results of the additional experiment with tool calling protocol.

Key Findings.

1. **ReAct benefits non-thinking models**: Every non-thinking model in the experiment benefits from using ReAct instead of native tool calling, while every thinking model does not.
2. **Diminishing returns**: From 6.4 percentage points for Llama-3.1-8B-Instruct to just 1.4 for

GPT-4.1, we consistently see diminishing returns from using ReAct.

7 Discussion

Benchmarking Agentic Capabilities. Our results demonstrate that FinAgent-Bench effectively discriminates between models' agentic capabilities. The significant gap from ceiling performance (around 69%) suggests current models have substantial room for improvement in financial tool use.

Open-Source Gap. The large gap between frontier and open-source models (up to 50+ percentage points) highlights an important direction for open-source development. Financial agentic tasks may require specialized training on tool-use trajectories.

Native Tool Calling vs. ReAct. ReAct allows the model to reason before choosing a tool to call. This has a large impact on performance for models that do not reason in their default setting, highlighting the improvement that test-time scaling can bring. For models that are already thinking, ReAct decreases performance. This may reflect that modern thinking models are well-optimized for structured function calling, while ReAct's verbose format introduces additional opportunities for error.

Quality Control. Our committee-based quality control ensures high-quality examples but introduces potential biases. The judges themselves are LLMs, which may share systematic blind spots with the evaluated models.

8 Conclusion

We introduced FinAgent-Bench, a benchmark for evaluating agentic LLMs on financial analysis tasks requiring tool use. Our benchmark includes two complementary datasets: SynthSQL with 5,979 SQL-based queries and GroundedFinQA with 1,108 multi-hop reasoning tasks. Evaluation of frontier and open-source models reveals significant room for improvement, with best accuracies of 69%.

Our work provides a foundation for developing and evaluating financial AI agents. Future work may extend the benchmark to additional financial domains, incorporate real corporate databases (with appropriate anonymization), and develop training approaches to improve agentic financial reasoning.

Limitations

- Synthetic data may not capture real-world financial complexity.
- Database schemas are generated, not derived from real companies.
- Evaluation is on English data only.
- The committee judges are LLMs themselves, and may share systematic blind spots with the evaluated models.
- The benchmark covers SQL-based and Python-based tool use; other tool modalities (e.g., web search, document retrieval) are not currently represented.

References

Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. 2025. Finance Agent Benchmark: Benchmarking llms on real-world financial research tasks. *Preprint*, arXiv:2508.00828.

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. FinQA: A dataset of numerical reasoning over financial data. *Proceedings of EMNLP 2021*.

Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. 2022. ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. *Proceedings of EMNLP 2022*.

Richard D Hipp. 2024. SQLite.

Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. 2023. Financebench: A new benchmark for financial question answering. *Preprint*, arXiv:2311.11944.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.

Fei Lei, Yibo Yang, Wenxiu Sun, and Dahua Lin. 2025. MCPVerse: An expansive, real-world benchmark for agentic tool use. *Preprint*, arXiv:2508.16260.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, and 1 others. 2024. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems*, 36.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2023. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv: 2308.03688*.

Akshara Prabhakar, Zuxin Liu, Ming Zhu, Jianguo Zhang, Tulika Awalgaonkar, Shiyu Wang, Zhiwei Liu, Haolin Chen, Thai Hoang, Juan Carlos Niebles, Shelby Heinecke, Weiran Yao, Huan Wang, Silvio Savarese, and Caiming Xiong. 2025. Apigen-mt: Agentic pipeline for multi-turn data generation via simulated agent-human interplay. *arXiv preprint arXiv:2504.03601*.

Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Yufei Huang, Chaojun Xiao, Chi Han, Yi Ren Fung, Yusheng Su, Huadong Wang, Cheng Qian, Runchu Tian, Kunlun Zhu, Shihao Liang, Xingyu Shen, Bokai Xu, and 22 others. 2023a. Tool learning with foundation models. *Preprint*, arXiv:2304.08354.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023b. Toolllm: Facilitating large language models to master 16000+ real-world apis. *Preprint*, arXiv:2307.16789.

Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. 2025. 'smolagents': a smol library to build great agentic systems. `https://github.com/huggingface/smolagents`.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Preprint*, arXiv:2302.04761.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. Self-Instruct: Aligning language model with self generated instructions.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Brussels, Belgium. Association for Computational Linguistics.

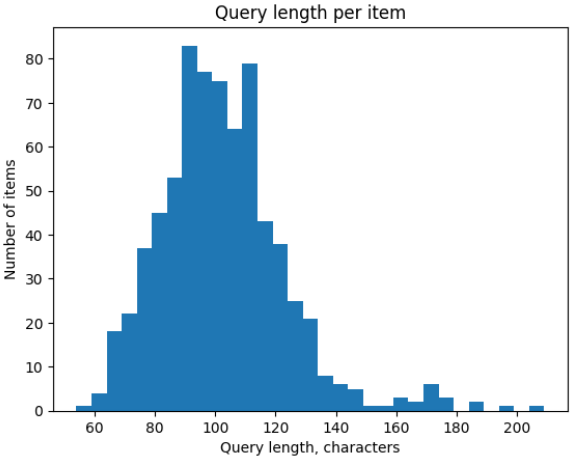

Figure 2: SynthSQL: Query length distribution.

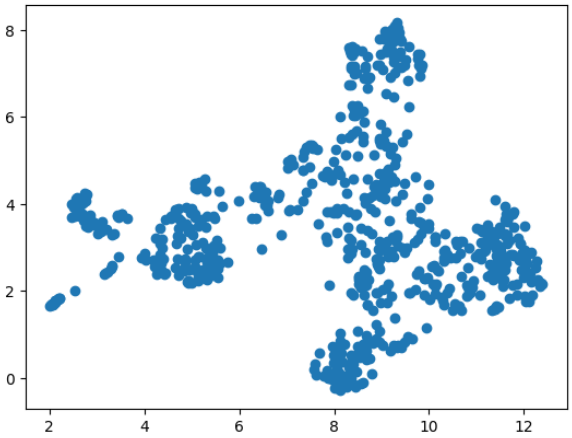

Figure 3: SynthSQL: UMAP of query embeddings.

Lingfeng Zeng, Fangqi Lou, Zixuan Wang, Jiajie Xu, Jinyi Niu, Mengping Li, Yifan Dong, Qi Qi, Wei Zhang, Ziwei Yang, Jun Han, Ruilun Feng, Ruiqi Hu, Lejie Zhang, Zhengbo Feng, Yicheng Ren, Xin Guo, Zhaowei Liu, Dongpo Cheng, and 2 others. 2025. FinGAIA: A chinese benchmark for ai agents in real-world financial domain. *Preprint*, arXiv:2507.17186.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3277–3287, Online. Association for Computational Linguistics.

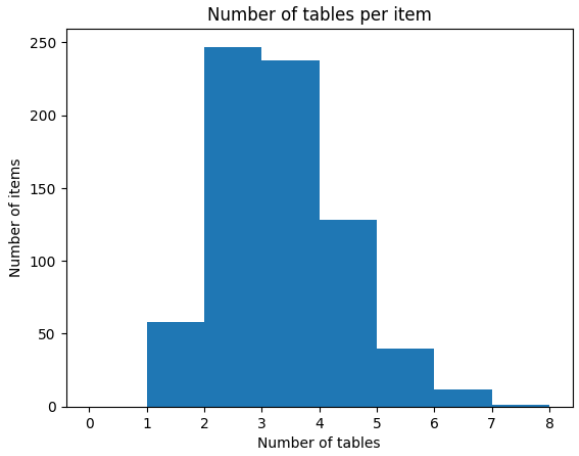

Figure 4: SynthSQL: Number of DB tables per example.

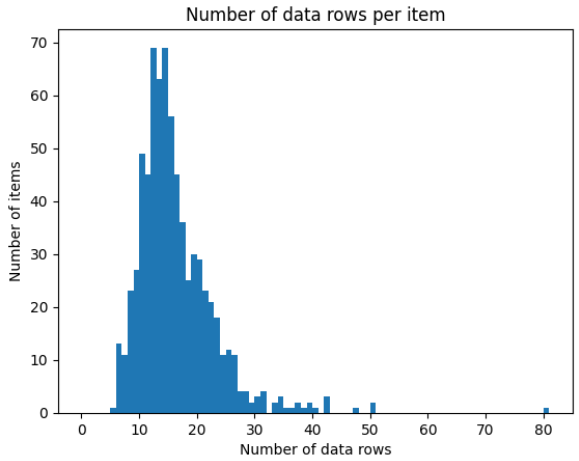

Figure 5: SynthSQL: Number of total DB rows per example.

A Dataset Statistics

A.1 SynthSQL Statistics

A.2 GroundedFinQA Statistics

B Prompts and Examples

Figure 13 shows the prompt template used for the committee LLM judge in SynthSQL.

Figure 14 shows an annotated example from the SynthSQL dataset illustrating the schema and data format.

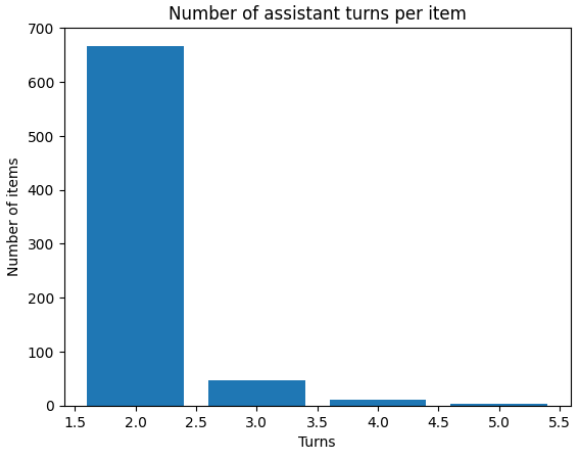

Figure 6: SynthSQL: Number of assistant turns per example.

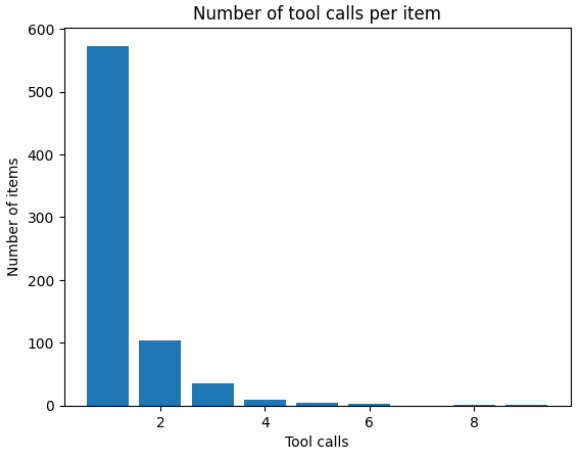

Figure 7: SynthSQL: Number of tool calls per example.

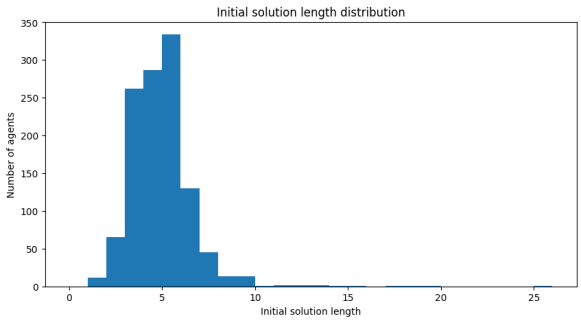

Figure 8: GroundedFinQA: Answer length distribution.

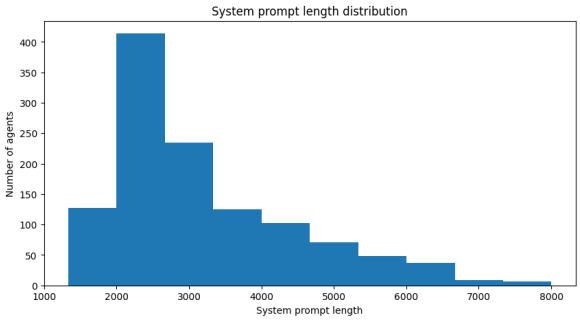

Figure 9: GroundedFinQA: System prompt length distribution.

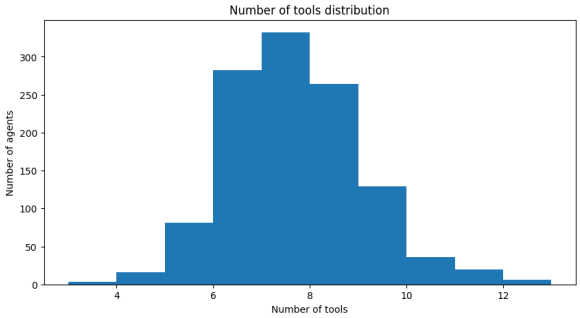

Figure 10: GroundedFinQA: Number of tools per example.

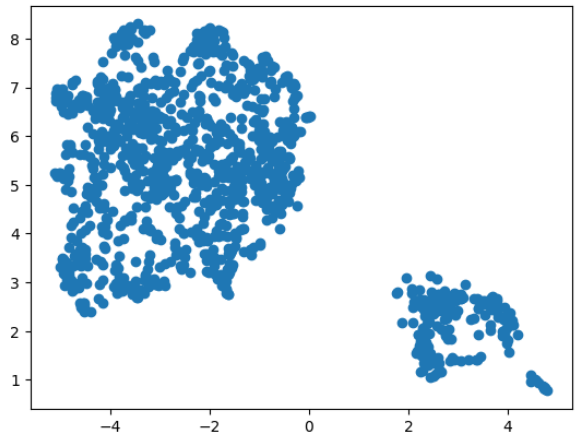

Figure 11: GroundedFinQA: UMAP of system prompt embeddings.

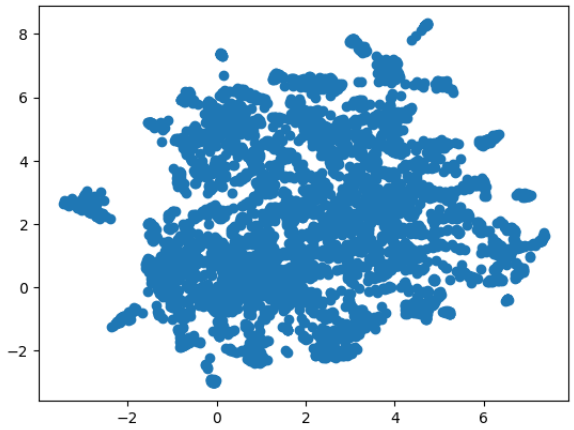

Figure 12: GroundedFinQA: UMAP of tool name embeddings.

```
# Task
You are a senior QA engineer working on AI agents for finance. You have to judge
the test case provided and check the following rules:

- The test case's data should be natural: it should be similar to what data could
  a real company have, and how it would be stored in a database.
- The agent's trace should be reasonable: the agent should look at the data needed
  to answer the query.
- The agent's trace should be sound: the reasoning steps should be logical, should
  follow from the previous steps and lead to the final answer.
- The agent's reasoning and final answer should be grounded: the only data the agent
  has access to is what was returned from its tool calls. The reasoning should not
  be based on any other data, including the data in the database that was not
  queried with the tools.
- The agent's final answer should answer the user's query.

In case one of these requirements are not met, you should provide a concise
actionable feedback so the author can improve the test case.

# Test case:

## Query
{{ item.query }}

## SQL schema
{{ item.schema_sql }}

## Data
{{ item.data_sql }}

## Agent trace
{{ item.agent_dialog | tojson(2) }}

# Output format
Reply with JSON object like this:
{
   "data_is_natural": 1 | 0,
   "trace_is_reasonable": 1 | 0,
   "trace_is_sound": 1 | 0,
   "reasoning_is_grounded": 1 | 0,
   "answer_is_sound": 1 | 0,
   "feedback": "... concise and actionable ..."
}
```

Figure 13: SynthSQL: Prompt template for a committee member judge.

```
Query:
Could duplicated invoice payments be causing discrepancies in our aged payables report?

Schema:
CREATE TABLE Suppliers (
    supplier_id INTEGER PRIMARY KEY,
    supplier_name TEXT NOT NULL
);
CREATE TABLE Invoices (
    invoice_id INTEGER PRIMARY KEY,
    supplier_id INTEGER NOT NULL,
    invoice_date DATE NOT NULL,
    due_date DATE NOT NULL,
    invoice_amount NUMERIC NOT NULL,
    FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id)
);
CREATE TABLE Payments (
    payment_id INTEGER PRIMARY KEY,
    invoice_id INTEGER NOT NULL,
    payment_date DATE NOT NULL,
    payment_amount NUMERIC NOT NULL,
    FOREIGN KEY (invoice_id) REFERENCES Invoices(invoice_id)
);

Data:
-- Invoice 1002 has duplicated payments covering entire invoice twice
INSERT INTO Payments VALUES
(5001, 1002, '2025-07-05', 500.00),
(5002, 1002, '2025-07-10', 500.00),
(5003, 1002, '2025-07-12', 500.00);  -- Duplicate over invoice amount
```

Figure 14: SynthSQL: Annotated example item showing query, schema, and data (excerpt).